Heaven's Light is Our Guide

Rajshahi University of Engineering & Technology

Department of Electrical and Computer Engineering

LAB REPORT-01

Course Code : ECE 2112
Course Title : Digital Techniques Sessional

Date of Submission: 26 July, 2026

Submitted by

Asrafun Naher Sarah
Roll: 2410017

Submitted to

Mst. Mazeda Noor Tasnim
Lecturer
Dept. of ECE, RUET

Task 1: OR Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate an OR gate using only NOR gates.
- To verify the constructed circuit against the standard OR gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate blocks, input switches, output indicator (LED).

Introduction

An OR gate gives a HIGH output whenever either of its two inputs is HIGH, and stays LOW only when both inputs are LOW. Since the NOR gate is known as a universal gate, meaning any logic function can be built using NOR gates alone, this experiment sets out to construct an OR gate purely from NOR gates and check that its behavior matches the standard OR truth table.

Theory

A NOR gate outputs the complement of what an OR gate would give for the same two inputs, so its output is defined as

$$Y = (A + B)'$$

A	B	$Y = (A + B)'$
0	0	1
0	1	0
1	0	0
1	1	0

Table 1: Truth Table of NOR Gate

If a signal is passed through a NOR gate twice, the double inversion cancels out and the original OR expression is recovered:

$$A + B = ((A + B)')'$$

This is the basis for the circuit built in this experiment – one NOR gate produces $(A+B)'$, and a second NOR gate, used purely as an inverter, turns that back into $A + B$.

Circuit Design

Inputs A and B were first connected to a NOR gate, whose output represents $(A + B)'$. This output was then fed into both inputs of a second NOR gate simultaneously, which made that gate behave as a simple inverter. The output taken from this second gate is therefore the final OR result, $A + B$.

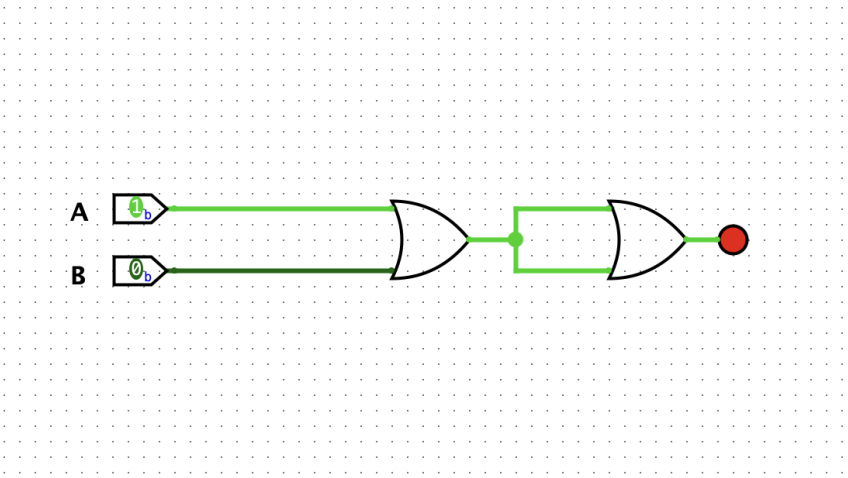

Figure 1: OR using NOR

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected OR gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Table 2: Testing of OR Gate using NOR Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the OR operation.

Conclusion

This experiment demonstrated that an OR gate can be fully realized using only NOR gates, since two NOR gates arranged in series – the second acting as an inverter – reproduce the OR function exactly. This confirms the universal nature of the NOR gate.

Task 2: OR Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate an OR gate using only NAND gates.
- To verify the constructed circuit against the standard OR gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate blocks, input switches, output indicator (LED).

Introduction

Like the NOR gate, the NAND gate is also classified as a universal gate, since any logic function – including OR – can be constructed using NAND gates exclusively. This experiment focuses on building an OR gate using only NAND gates, relying on De Morgan's theorem to convert between the two forms.

Theory

A NAND gate outputs the complement of what an AND gate would give for the same two inputs:

$$Y = (A \cdot B)'$$

A	B	$Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

Table 3: Truth Table of NAND Gate

By De Morgan's theorem, the OR operation can be rewritten entirely in terms of complemented AND:

$$A + B = (A' \cdot B')'$$

This means that if A' and B' are generated first, and then combined through a NAND operation, the result is exactly $A + B$. Since a NAND gate with both its inputs tied together behaves as a plain inverter, this entire expression can be realized using NAND gates alone.

Circuit Design

Two NAND gates were first configured as inverters – one for input A and one for input B – by connecting each gate's own input to both of its terminals, producing A' and B' respectively. These two inverted signals were then fed into a third NAND gate. Since this third gate receives A' and B', its output corresponds to $(A' \cdot B')'$, which by De Morgan's theorem is equivalent to $A + B$, giving the final OR result.

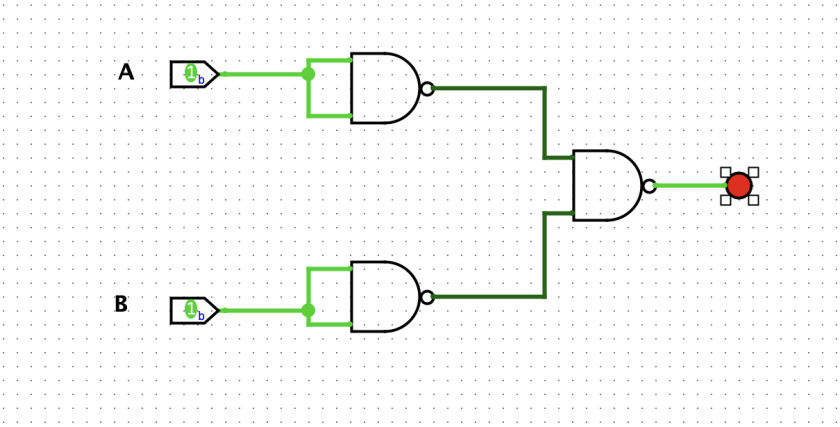

Figure 2: OR using NAND

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected OR gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	1

Table 4: Testing of OR Gate using NAND Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the OR operation.

Conclusion

This experiment demonstrated that an OR gate can be fully realized using only NAND gates, by first generating A' and B' through two NAND gates wired as inverters, and then combining them through a third NAND gate. This confirms, by application of De Morgan's theorem, that the NAND gate is a universal gate capable of producing the OR function.

Task 3: AND Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate an AND gate using only NOR gates.
- To verify the constructed circuit against the standard AND gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate blocks, input switches, output indicator (LED).

Introduction

Having already used NOR gates to build an OR gate, this experiment extends the idea further by constructing an AND gate from NOR gates alone. Since the NOR gate is universal, this is achievable through De Morgan's theorem, which relates AND and OR operations through complementation.

Theory

An AND gate produces a HIGH output only when both of its inputs are HIGH:

$$Y = A \cdot B$$

A	B	$Y = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

Table 5: Truth Table of AND Gate

By De Morgan's theorem, the AND operation can be expressed entirely through complemented OR:

$$A \cdot B = (A' + B')'$$

This means that if A' and B' are produced first, and then combined through a NOR operation, the result is exactly $A \cdot B$. Since tying both inputs of a NOR gate together makes it behave as a simple inverter, this whole expression can be built using NOR gates only.

Circuit Design

Two NOR gates were first set up as inverters – one for input A and one for input B – by connecting each gate's own input to both of its terminals, producing A' and B' respectively. These two inverted signals were then passed into a third NOR gate. Since this third gate receives A' and B', its output corresponds to $(A' + B')'$, which by De Morgan's theorem is equivalent to $A \cdot B$, giving the final AND result.

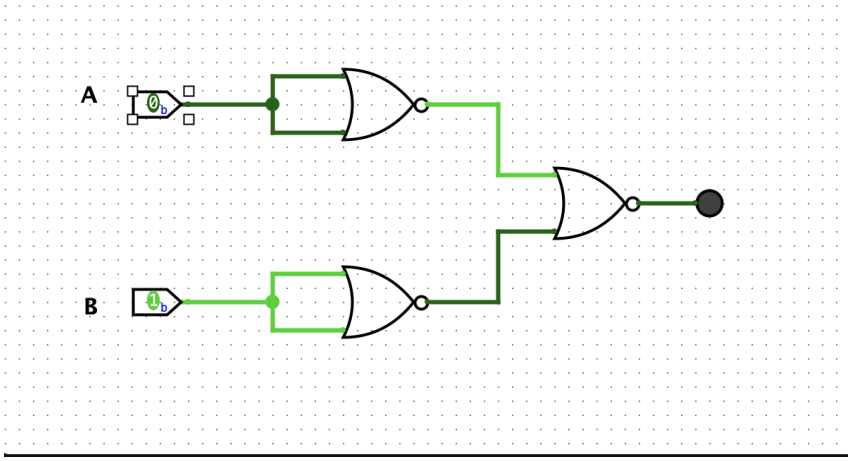

Figure 3: AND using NOR

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected AND gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Table 6: Testing of AND Gate using NOR Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the AND operation.

Conclusion

This experiment demonstrated that an AND gate can be fully realized using only NOR gates, by first generating A' and B' through two NOR gates wired as inverters, and then combining them through a third NOR gate. This confirms, by application of De Morgan's theorem, that the NOR gate is a universal gate capable of producing the AND function.

Task 4: AND Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate an AND gate using only NAND gates.
- To verify the constructed circuit against the standard AND gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate blocks, input switches, output indicator (LED).

Introduction

Since a NAND gate is essentially an AND gate followed by an inverter, recovering a pure AND gate from NAND gates is a matter of cancelling out that inversion. This experiment builds an AND gate using two NAND gates connected in series and verifies its output against the standard AND truth table.

Theory

A NAND gate produces the complement of what an AND gate would output for the same two inputs:

$$Y = (A \cdot B)'$$

A	B	$Y = (A \cdot B)'$
0	0	1
0	1	1
1	0	1
1	1	0

Table 7: Truth Table of NAND Gate

Passing a signal through a NAND gate twice undoes the inversion and recovers the original AND expression:

$$A \cdot B = ((A \cdot B)')'$$

This means the first NAND gate can be used to generate $(A \cdot B)'$ from inputs A and B, and a second NAND gate, wired so both of its terminals receive that same signal, acts purely as an inverter and restores the AND result.

Circuit Design

Inputs A and B were first applied to a NAND gate, producing $(A \cdot B)'$ at its output. This signal was then fed into both inputs of a second NAND gate simultaneously, causing that

gate to function as a simple inverter. The output taken from this second gate is therefore the final AND result, $A \cdot B$.

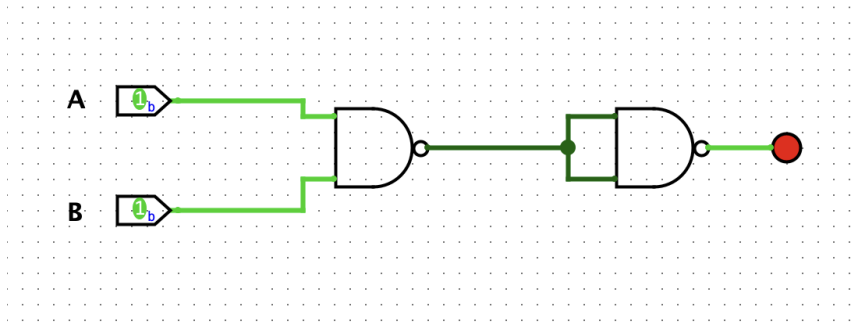

Figure 4: AND using NAND

Result and Discussion

The circuit was tested for all four possible combinations of A and B, and the output obtained in each case was compared against the expected AND gate output.

A	B	Expected Output	Obtained Output
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

Table 8: Testing of AND Gate using NAND Gates

In every case, the obtained output matched the expected result, confirming that the circuit correctly performs the AND operation.

Conclusion

This experiment demonstrated that an AND gate can be fully realized using only NAND gates, since two NAND gates in series – the second acting as an inverter – reproduce the AND function exactly. This confirms the universal nature of the NAND gate.

Task 5: NOT Gate Implementation using NOR Gates

Objectives

- To understand the working principle of a NOR gate.
- To design and simulate a NOT gate using only a NOR gate.
- To verify the constructed circuit against the standard NOT gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NOR gate block, input switch, output indicator (LED).

Introduction

Unlike the previous experiments, which combined two inputs, a NOT gate works on a single input and simply inverts it. This experiment shows that a NOR gate, when both of its inputs are tied to the same signal, reduces to exactly this behavior – functioning as a plain inverter.

Theory

A NOT gate outputs the complement of its single input:

$$Y = A'$$

A	$Y = A'$
0	1
1	0

Table 9: Truth Table of NOT Gate

If both inputs of a NOR gate are connected to the same signal A, its output reduces to

$$A' = A \text{ NOR } A$$

which is precisely the NOT operation. This works because the NOR expression $(A + A)'$ simplifies to A', since $A + A = A$.

Circuit Design

A single input, A, was connected to both terminals of a NOR gate simultaneously. With both inputs identical, the gate's OR stage produces A internally, which is then inverted by the NOR gate's built-in complement stage, so the final output observed is A'.

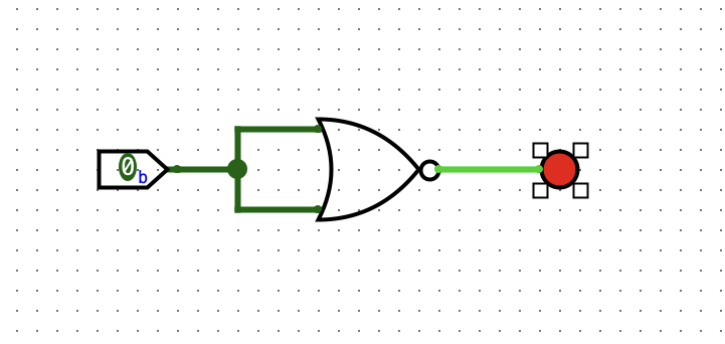

Figure 5: NOT using NOR

Result and Discussion

The circuit was tested for both possible values of A, and the output obtained in each case was compared against the expected NOT gate output.

A	Expected Output	Obtained Output
0	1	1
1	0	0

Table 10: Testing of NOT Gate using NOR Gate

In both cases, the obtained output matched the expected result, confirming that the circuit correctly performs the NOT operation.

Conclusion

This experiment demonstrated that a NOR gate can be reduced to a NOT gate simply by connecting both of its inputs to the same signal. This further confirms the universal switching capability of the NOR gate, since it can realize even the most basic logic operation.

Task 6: NOT Gate Implementation using NAND Gates

Objectives

- To understand the working principle of a NAND gate.
- To design and simulate a NOT gate using only a NAND gate.
- To verify the constructed circuit against the standard NOT gate truth table.

Apparatus / Tools Used

Digital logic circuit simulation software, NAND gate block, input switch, output indicator (LED).

Introduction

This experiment completes the set of single-gate inverter constructions by showing that, just like the NOR gate, a NAND gate can also be reduced to a NOT gate. Tying both of its inputs to the same signal collapses its two-input behavior down to a simple inversion.

Theory

A NOT gate outputs the complement of its single input:

$$Y = A'$$

A	$Y = A'$
0	1
1	0

Table 11: Truth Table of NOT Gate

If both inputs of a NAND gate are connected to the same signal A, its output reduces to

$$A' = A \text{ NAND } A$$

which is precisely the NOT operation. This follows because the NAND expression $(A \cdot A)'$ simplifies to A', since $A \cdot A = A$.

Circuit Design

A single input, A, was connected to both terminals of a NAND gate simultaneously. With both inputs identical, the gate's internal AND stage produces A, which is then inverted by the NAND gate's complement stage, so the final output observed is A'.

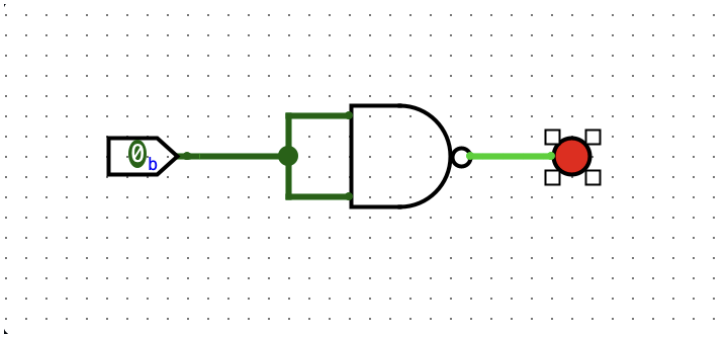

Figure 6: NOT using NAND

Result and Discussion

The circuit was tested for both possible values of A, and the output obtained in each case was compared against the expected NOT gate output.

A	Expected Output	Obtained Output
0	1	1
1	0	0

Table 12: Testing of NOT Gate using NAND Gate

In both cases, the obtained output matched the expected result, confirming that the circuit correctly performs the NOT operation.

Conclusion

This experiment demonstrated that a NAND gate can be reduced to a NOT gate simply by connecting both of its inputs to the same signal. Together with the previous experiment, this confirms that both the NOR and NAND gates are capable of realizing the NOT operation, reinforcing their status as universal gates.

Task 7: Implementation of Full Adder and Testing

Objectives

- To understand the working principle of a Full Adder circuit.
- To construct a Full Adder using two Half Adders and an OR gate.
- To verify the constructed circuit against the standard Full Adder truth table for all input combinations.

Apparatus / Tools Used

Digital logic circuit simulation software, XOR gates, AND gates, OR gate, input switches, output indicators (LEDs).

Introduction

Unlike the previous experiments, which dealt with single logic gates, this experiment focuses on a combinational circuit built from multiple gates working together. A Full Adder extends the idea of a Half Adder by accepting a third input – a carry-in – so that multiple bits can be added in sequence, as required in multi-bit binary addition. This experiment constructs a Full Adder from two Half Adder blocks and verifies its two outputs, Sum and Carry-out, across all eight possible input combinations.

Theory

A Full Adder takes three single-bit inputs, A, B, and a carry-in (C_{in}), and produces two outputs: the Sum and the carry-out (C_{out}). These are expressed as

$$Sum = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + C_{in}(A \oplus B)$$

Looking at the Sum expression, it can be seen that it is really just an XOR of three signals, which can be built by cascading two Half Adders – the first combining A and B, and the second combining that result with C_{in}. The carry-out, meanwhile, is HIGH whenever either Half Adder produces a carry, so the two individual carry signals can simply be OR-ed together to obtain the final C_{out}.

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Table 13: Truth Table of Full Adder

Circuit Design

The first Half Adder was built to accept inputs A and B, producing a partial sum along with its own carry output. This partial sum was then passed, along with C_{in}, into a second Half Adder, which produced the final Sum output directly. The carry outputs from both Half Adders were connected to the two inputs of an OR gate, and the output of this OR gate was taken as the final Carry-out, C_{out}, of the Full Adder.

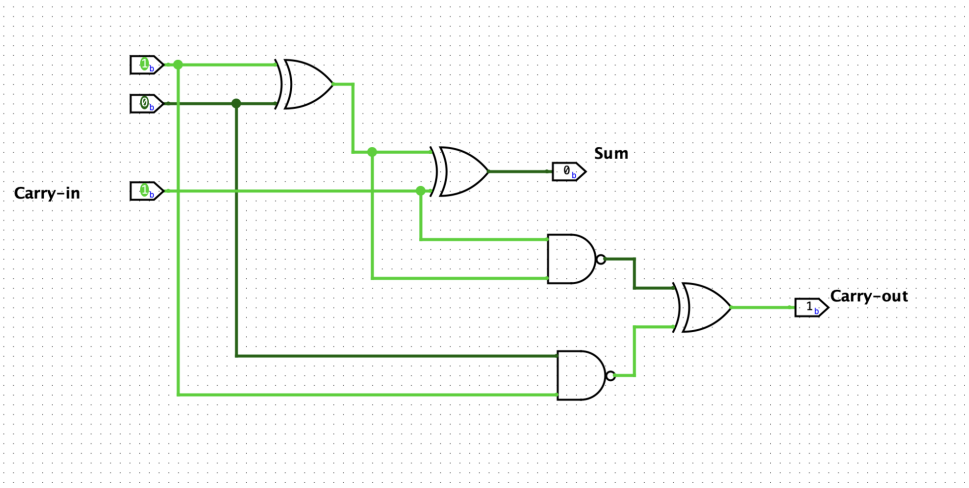

Figure 7: Full Adder circuit

Result and Discussion

Since a Full Adder has three inputs, it was tested across all eight possible combinations of A, B, and C_{in}, with both the Sum and Carry-out outputs recorded and compared against the expected values.

For every one of the eight input combinations, the obtained Sum and Carry-out values matched the expected results exactly, confirming that the circuit correctly implements Full Adder behavior.

A	B	C_{in}	Expected (Sum, C_{out})	Obtained (Sum, C_{out})
0	0	0	0, 0	0, 0
0	0	1	1, 0	1, 0
0	1	0	1, 0	1, 0
0	1	1	0, 1	0, 1
1	0	0	1, 0	1, 0
1	0	1	0, 1	0, 1
1	1	0	0, 1	0, 1
1	1	1	1, 1	1, 1

Table 14: Testing of Full Adder

Conclusion

This experiment demonstrated that a Full Adder can be successfully built by cascading two Half Adders and combining their carry outputs through an OR gate. The verified results across all eight input combinations confirm correct three-bit binary addition, forming the basis for larger multi-bit adders.

Task 8: Implementation of Binary to BCD Converter

Objectives

- To understand the concept of Binary-Coded Decimal (BCD) representation.
- To design and simulate a 4-bit Binary to BCD Converter circuit.
- To verify the circuit's output against the expected BCD values for various binary inputs.

Apparatus / Tools Used

Digital logic circuit simulation software, Binary-to-BCD converter block, input switches, seven-segment BCD display.

Introduction

Digital circuits naturally work in binary, but decimal is easier for humans to read – for example, on a seven-segment display. This experiment implements a converter that takes a 4-bit binary input and re-expresses it in Binary-Coded Decimal (BCD) form.

Theory

A 4-bit binary input can represent values 0 to 15. For values 0–9, the BCD output matches the binary input directly, since one decimal digit fits in 4 bits. For values 10 and above, the number is split across two BCD digit groups – one per decimal digit. For example,

$$1010_2 = 10_{10} \rightarrow 0001\ 0000_{\text{BCD}}$$

The complete mapping for all 16 possible inputs is shown below.

B3	B2	B1	B0	D4	D3	D2	D1
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	0
0	0	1	1	0	0	1	1
0	1	0	0	0	1	0	0
0	1	0	1	0	1	0	1
0	1	1	0	0	1	1	0
0	1	1	1	0	1	1	1
1	0	0	0	1	0	0	0
1	0	0	1	1	0	0	1
1	0	1	0	1	0	1	0
1	0	1	1	1	0	1	1
1	1	0	0	1	1	0	0
1	1	0	1	1	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	1	1	1	1

Table 15: Truth Table of Binary to BCD Converter

Circuit Design

The four binary inputs, B3 through B0, were connected to the Binary-to-BCD Converter block, which internally checks whether the value exceeds 9 and splits it across two BCD digit groups if needed. The resulting BCD output was connected to a seven-segment display to read off the corresponding decimal digit(s).

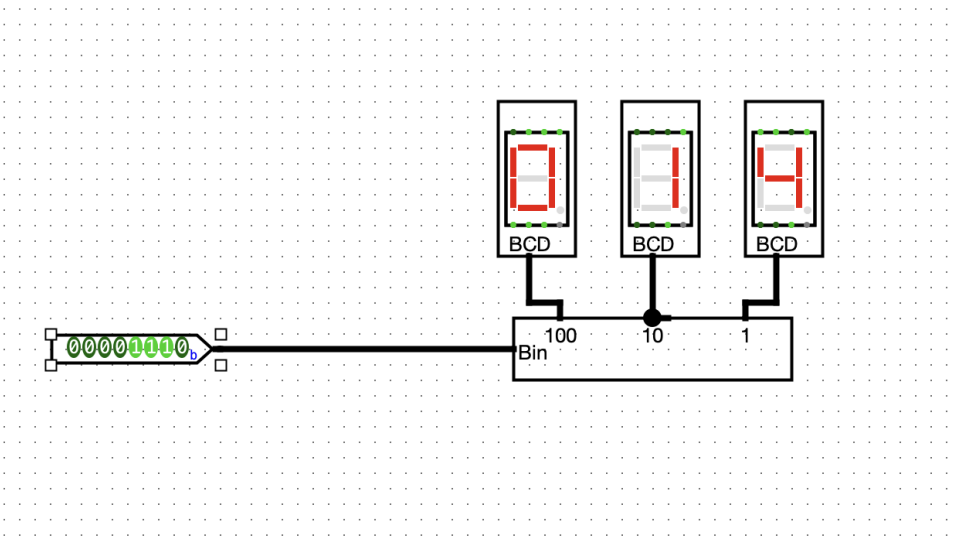

Figure 8: Binary to BCD

Result and Discussion

A range of binary inputs, including values both below and above 9, were applied to the converter, and the resulting BCD output was compared against the expected decimal equivalent in each case.

Binary Input	Decimal Equivalent	BCD Output
0000	0	0000
0001	1	0001
0101	5	0101
1001	9	1001
1010	10	0001 0000
1111	15	0001 0101

Table 16: Testing of Binary to BCD Converter

The observed BCD outputs matched the expected decimal values across all tested inputs, including the cases where the value exceeded 9 and had to be split across two BCD digit groups.

Conclusion

This experiment demonstrated that a 4-bit binary number can be successfully converted into its Binary-Coded Decimal equivalent, with values above 9 correctly redistributed across two separate digit groups. The verified results confirm the practical role such converters play in driving decimal-based displays from binary digital circuits.

References

1. **M. M. Mano**, Digital Design, 5th ed. Upper Saddle River, NJ: Pearson, 2013.

2. **R. J. Tocci, N. S. Widmer, and G. L. Moss**, Digital Systems: Principles and Applications, 11th ed. Upper Saddle River, NJ: Pearson, 2011.

3. **Department of Electrical and Computer Engineering, RUET**, ECE 2112: Digital Techniques Sessional – Course Materials, 2026.

4. **Logisim-evolution**, Digital Logic Design Simulator, Documentation. Available: `https://github.com/logisim-evolution/logisim-evolution`